# DIAGRAMA UML — SIMULADOR DE MISIÓN ESPACIAL

Ejercicio 5 · Polimorfismo vía herencia · Clases y miembros declarados en src

## Main

Sin atributos declarados

+ main(args: String[]): void

«create» / inicia

## SimuladorController

- mision: Mision
- view: SimuladorView

+ SimuladorController(mision: Mision, view: SimuladorView)
+ listarModulos(): void
+ consultarCatalogoPorCosto(): void
+ consultarResumenComunicaciones(): void
+ avanzarCiclo(): void
+ buscarModulo(): void
+ iniciarSimulador(): void

view

## SimuladorView

- scanner: Scanner

+ SimuladorView()
+ mostrarMenuPrincipal(): void
+ leerString(mensaje: String): String
+ leerEntero(mensaje: String): int
+ leerDouble(mensaje: String): double
+ leerOpcion(): int
+ mostrarMensaje(mensaje: String): void
+ mostrarModulo(modulo: Modulo): void
+ mostrarModulos(modulos: List<Modulo>): void
+ mostrarCatalogo(modulos: List<Modulo>): void
+ mostrarResultadosCiclo(resultados: List<String>): void
+ mostrarEstadoMision(mision: Mision): void
+ mostrarResumenComunicaciones(mision: Mision): void

mision

## Mision

- modulos: List<Modulo>
- energiaDisponible: double
- datosEnOrbita: double
- datosDescargados: double
- ciclosTranscurridos: int

+ Mision(energiaDisponible: double)
+ getModulos(): List<Modulo>
+ getEnergiaDisponible(): double
+ getDatosEnOrbita(): double
+ getDatosDescargados(): double
+ getCiclosTranscurridos(): int
+ agregarModulo(modulo: Modulo): void
+ agregarEnergia(cantidad: double): void
+ consumirEnergia(cantidad: double): boolean
+ agregarDatosEnOrbita(cantidad: double): void
+ descargarDatos(cantidad: double): boolean
+ buscarModuloPorId(id: int): Modulo
+ buscarModuloPorNombre(nombre: String): Modulo
+ avanzarCiclo(): List<String>
+ obtenerModulosTierra(): List<ModuloTierra>
+ contarModulosTierra(): int
+ contarModulosTierraActivos(): int
+ obtenerCatalogoPorCosto(): List<Modulo>
+ calcularCapacidadDescargaActiva(): double
+ obtenerMayoresDescargadores(): List<ModuloTierra>

modulos 0..*

{abstract}

## *Modulo*

# id: int
# nombre: String
# salud: double
# activo: boolean
# costoConstruccion: double

+ Modulo(id: int, nombre: String, salud: double, activo: boolean, costoConstruccion: double)
+ getId(): int
+ getNombre(): String
+ getSalud(): double
+ isActivo(): boolean
+ getCostoConstruccion(): double
+ setSalud(salud: double): void
+ setActivo(activo: boolean): void
+ *procesarCiclo(mision: Mision): String*
+ toString(): String

**Notación UML**
+ público  − privado  # protegido
Cursiva: clase o método abstracto
Subrayado: método estático
Triángulo vacío: herencia hacia Modulo
Flecha continua: asociación navegable
Flecha discontinua: dependencia

**Polimorfismo**
Mision.avanzarCiclo() recorre List<Modulo> e invoca modulo.procesarCiclo(this). Cada subclase ejecuta su propia implementación.

Los módulos reciben Mision como parámetro; no la conservan como atributo.

Los miembros heredados no se repiten. Las dependencias adicionales se muestran en la segunda página.

## ModuloEnergia

- energiaPorCiclo: double
- ciclosAcumulados: int
- totalGenerado: double

+ ModuloEnergia(id: int, nombre: String, salud: double, activo: boolean, costoConstruccion: double, energiaPorCiclo: double)
+ getEnergiaPorCiclo(): double
+ getCiclosAcumulados(): int
+ getTotalGenerado(): double
+ procesarCiclo(mision: Mision): String
+ toString(): String

## ModuloVuelo

- tipoInstrumento: String
- datosPorCiclo: double
- consumoEnergetico: double
- ciclosAcumulados: int
- totalAcumulado: double

+ ModuloVuelo(id: int, nombre: String, salud: double, activo: boolean, costoConstruccion: double, tipoInstrumento: String, datosPorCiclo: double, consumoEnergetico: double)
+ getTipoInstrumento(): String
+ getDatosPorCiclo(): double
+ getConsumoEnergetico(): double
+ getCiclosAcumulados(): int
+ getTotalAcumulado(): double
+ procesarCiclo(mision: Mision): String
+ toString(): String

## ModuloTierra

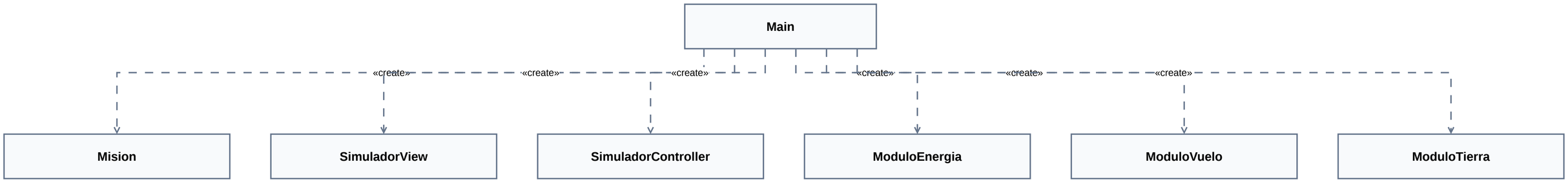
- estacion: String
- capacidadDescarga: double
- energiaPorDescarga: double
- totalDescargado: double

+ ModuloTierra(id: int, nombre: String, salud: double, activo: boolean, costoConstruccion: double, estacion: String, capacidadDescarga: double, energiaPorDescarga: double)
+ getEstacion(): String
+ getCapacidadDescarga(): double
+ getEnergiaPorDescarga(): double
+ getTotalDescargado(): double
+ procesarCiclo(mision: Mision): String
+ toString(): String

# DEPENDENCIAS Y COLABORACIÓN

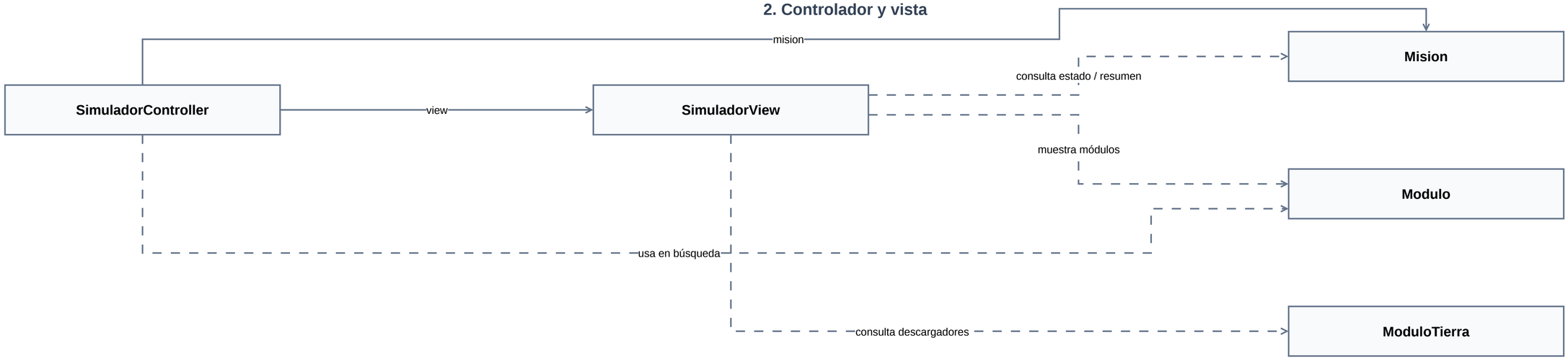
Vistas complementarias · Se omiten clases de biblioteca como Scanner, List y ArrayList

## 1. Inicialización desde Main

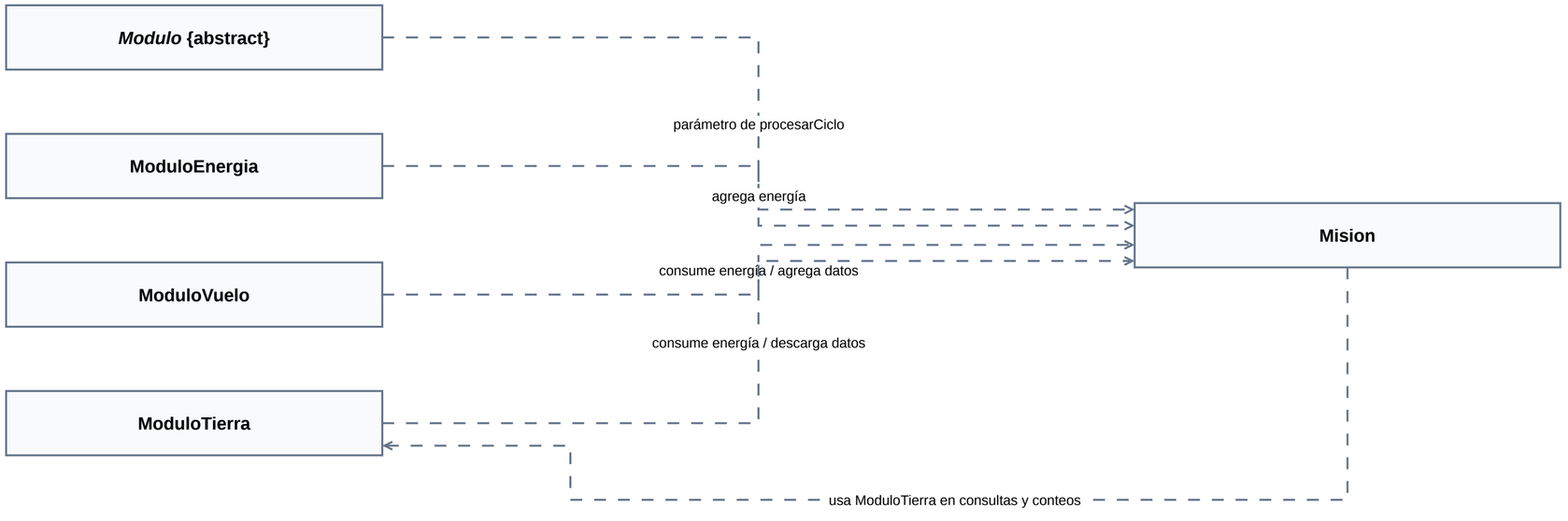



Main también utiliza Modulo como tipo del arreglo inicial y del recorrido que registra los módulos.

## 2. Controlador y vista




## 3. Modelo y procesamiento por ciclo




**Asociación y dependencias**
Mision conserva 0..* módulos. Su relación con Modulo está en la primera página.

Las flechas discontinuas representan uso temporal de un tipo o sus operaciones, no atributos adicionales.

Los objetos se construyen fuera del controlador y de Mision; no se presupone composición ni propiedad exclusiva.